

Research Report: Optimization of the Swaram Speech-to-Text Pipeline for Indian Locales

Executive Summary

An exhaustive analysis of the Swaram application’s existing technical architecture, coupled with a rigorous evaluation of the 2026 state-of-the-art in automatic speech recognition (ASR), voice-agent pipelines, and multimodal large language models, reveals substantial opportunities to optimize the platform for latency, proper-noun accuracy, and cost-efficiency. The current Swaram architecture relies heavily on discrete, clip-based REST API interactions driven by a primitive root-mean-square (RMS) energy Voice Activity Detection (VAD) implementation. This sequential paradigm limits system responsiveness and fundamentally restricts the application from delivering a fluid, conversational user experience comparable to leading global voice agents. Furthermore, the system’s reliance on specific legacy fallbacks—most notably on-device and cloud Whisper models for all languages—contradicts recent empirical performance benchmarks for Indian languages, particularly Dravidian script families. The following represent the five highest-leverage architectural modifications for the Swaram engineering team, ranked by the designated formula: $(\text{Accuracy Gain} \times \text{Latency Gain}) / (\text{Cost} \times \text{Implementation Effort})$.

Rank	Architectural Modification	Primary Impact Domain	Target Dossier Section	Rationale and Projected Return on Investment
1	Migrate to Sarvam WebSocket with Semantic Endpointing	Latency and Interaction Fluidity	§5.2, §6.2, §11 (W6, W7)	Replaces fixed 500ms acoustic silence delays and complete REST round-trips with continuous audio streaming and LLM-driven semantic turn detection. This

				delivers a massive latency reduction at identical per-second Sarvam processing costs, creating an "instant" conversational feel.
2	Activate Gemini 3.1 Flash-Lite Audio in the Accuracy Ensemble	Accuracy (Proper Nouns and Named Entities)	§6.2, §11 (W1, W3)	Gemini 3.1 Flash-Lite costs just \$0.50 per 1 million audio tokens and excels at multimodal reasoning. It consistently outperforms standard STT pipelines on proper nouns when supplied with explicit context variables directly in the multimodal prompt.
3	Implement Silero VAD with ort-wasm iOS Safari Fallback	Robustness and Audio Capture	§5.2, §11 (W7, W8)	Replaces the deprecated, main-thread-blocking ScriptProcessorNode.

				Resolves critical iOS Safari WebAssembly SIMD limitations via a strictly configured fallback ONNX runtime, preventing the premature clipping of soft consonants that triggers downstream hallucinations.
4	Deploy Bulbul v3 dict_id for English Readback Sequences	Text-to-Speech Prosody	§8.1, §11 (W15, W16)	Eliminates the need for crude, error-prone transliteration tricks by injecting exact Indian-name phonetic pronunciation dictionaries directly into the Sarvam TTS engine via a pre-compiled JSON payload.
5	Transition from Simple Deduplication to Generative Error Correction	Accuracy and Hypothesis Fusion	§6.2, §11 (W1, W2)	Adapts the academic Whispering-LLaMA paradigm. Instead of simple

				hypothesis deduplication, the Groq-powered fusion LLM interprets phonetic alignments across differing engine outputs to reconstruct the correct Indian name dynamically.
--	--	--	--	--

The following report systematically breaks down the theoretical foundations, implementation parameters, and comparative benchmark data required to execute these optimizations within the constraints of a Next.js Progressive Web App operating on low-end mobile hardware.

1. Streaming and Latency: Architecting a Real-Time STT Pipeline

To achieve conversational responsiveness where the time-to-first-token (TTFT) drops below the 500ms threshold, the voice pipeline must transition from a sequential batch-processing model to an overlapping, streaming architecture. In the current iteration, Swaram waits for absolute acoustic silence, finalizes an audio blob, and dispatches it over a REST POST request. This creates a hard latency floor dictated by the sum of the VAD timeout, the network round-trip, and the transcription processing time. Production voice agents such as LiveKit, Pipecat, and OpenAI Realtime eliminate this bottleneck by overlapping the pipeline stages: VAD, Speech-to-Text, LLM generation, and Text-to-Speech occur concurrently.¹

1.1 Vendor Streaming Capabilities, Protocols, and Pricing

The transition to a streaming architecture requires a deep understanding of the transport protocols and partial-result semantics exposed by Swaram's primary vendors.

Sarvam Saaras v3 WebSocket Streaming Sarvam provides a highly capable, persistent WebSocket streaming endpoint (`wss://api.sarvam.ai/speech-to-text/ws`) heavily optimized for Indian linguistic structures.³

- **Protocol Mechanics:** Authentication is handled during the initial HTTP GET upgrade via the `Api-Subscription-Key` header.³ The client then transmits an initial JSON configuration payload defining parameters such as the `language-code`, `model` (defaulting to `saaras:v3`),

and the operational mode (e.g., transcribe, codemix, or verbatim).³ Audio data is subsequently streamed in JSON wrappers (e.g., {"audio": {"data": "<base64_encoded_audio>", "sample_rate": "16000", "encoding": "audio/wav"}}).³

- **Partial Result Semantics:** The endpoint asynchronously emits interim hypotheses as the user speaks. The response schema utilizes a strictly typed JSON structure: {"type": "data", "data": {"request_id": "...", "transcript": "...", "metrics": {"audio_duration": 1.1, "processing_latency": 1.1}}}.³ This allows the user interface to render recognizing text instantly.
- **Endpointing Controls:** Unlike the REST API, the WebSocket connection exposes deep server-side VAD tuning parameters, including positive_speech_threshold (default 0.7), negative_speech_threshold (default 0.45), and negative_frames_count (default 18 frames).³ Furthermore, the client can force an immediate transcription finalization by sending a {"flush_signal": true} object.³
- **Pricing Strategy:** Sarvam bills streaming connections at ₹30 per hour of processed audio, maintaining strict cost parity with their batch REST endpoint, meaning the latency upgrade incurs no financial penalty.⁴

Azure Speech Streaming and Fast Transcription

Azure offers multiple ingestion methodologies with distinct pricing and latency profiles.

- **Streaming SDK:** True streaming requires the microsoft-cognitiveservices-speech-sdk. The SDK utilizes GStreamer under the hood to manage binary audio format decompression before managing a WebRTC/WebSocket upgrade utilizing the 101 Switching Protocols handshake.⁶ This is billed at \$1.00 per hour for standard real-time transcription.⁸
- **Fast Transcription REST API:** Azure has recently introduced a "Fast Transcription" endpoint designed for predictable latency on short audio segments.¹⁰ While this is a synchronous REST API and does not emit interim partials, it processes audio significantly faster than real-time speed. It is highly cost-effective at \$0.66 per hour.¹⁰

Groq Whisper (large-v3-turbo) Limitations Groq provides access to OpenAI's Whisper large-v3-turbo model, which operates at a highly disruptive price point of \$0.04 per hour of audio.¹² The model achieves processing speed factors between 216x and 228x faster than real-time.¹³ However, the critical limitation of the Groq implementation is the absence of a native WebSocket streaming API.¹³ It strictly processes chunked audio files via standard REST payloads. Attempting to simulate a streaming experience with Groq requires aggressive client-side audio chunking and overlapping REST calls, which introduces massive orchestration complexity and severely degrades Word Error Rate (WER) at the artificial chunk boundaries. Therefore, Groq is fundamentally unsuitable for the real-time interim-results portion of the pipeline.

1.2 Target Streaming Architecture for the Next.js PWA

To integrate sub-second streaming into Swaram's PWA architecture without compromising the fallback chain, the system must adopt a "Half-Cascade Streaming Architecture." This design routes the everyday, single-engine transcription path through the streaming interface while

preserving the batch-based ensemble for the highly critical proper-noun accuracy path.

1. **AudioWorklet Ingestion:** The browser utilizes an AudioWorklet to capture raw PCM audio from the user's microphone, bypassing the main-thread latency inherent to the deprecated ScriptProcessorNode.
2. **Continuous WebSocket Proxy:** A dedicated Next.js API route (/api/transcribe/stream) establishes a persistent connection to the Sarvam WebSocket endpoint. The browser streams base64-encoded PCM frames to this proxy, which relays them to Sarvam.
3. **Interim Emission and Rendering:** As Sarvam emits partial transcripts, the Next.js proxy relays them to the client via Server-Sent Events (SSE) or a direct WebSocket, allowing the PWA to update the visual dialogue state machine instantaneously.
4. **Graceful Degradation:** If the WebSocket connection fails at any point in the proxy chain, the AudioWorklet is instructed to retain its audio buffer, finalize the blob, and immediately fall back to the existing batch REST API (/api/transcribe), ensuring no loss of user input.

2. Voice Activity Detection and Semantic Endpointing

The dossier indicates that Swaram currently employs a root-mean-square (RMS) energy-threshold VAD implementation. Energy-based VADs are fundamentally flawed for accessibility applications; they are easily triggered by ambient noise (such as ceiling fans or traffic) and routinely clip the soft phonetic onsets of Indian names, directly causing downstream hallucinations.

2.1 Implementing Silero VAD via WebAssembly

The industry consensus for optimal browser-side acoustic VAD is the deployment of lightweight neural networks, specifically the Silero VAD model running via the @ricky0123/vad-web library.¹⁵ This architecture completely abandons energy heuristics in favor of predicting the statistical probability of human speech presence.

Mitigating iOS Safari WebAssembly Constraints The dossier imposes a hard constraint requiring support for low-end Android devices and iOS Safari. A significant architectural hurdle exists when deploying neural VADs on Apple platforms. Recent iterations of Safari (specifically within the iOS 16 and iOS 17 ecosystems) lack comprehensive support for WebAssembly Single Instruction, Multiple Data (SIMD) execution.¹⁶ Attempting to load the standard SIMD-optimized onnxruntime-web binary results in a fatal environment error: [wasm] Error: WebAssembly SIMD is not supported in the current environment.¹⁶

To ensure universal compatibility without sacrificing the neural VAD framework, the Swaram implementation must dynamically configure the ONNX execution provider. The system must probe for SIMD support upon initialization; if absent, it must explicitly load a legacy, non-SIMD WebAssembly binary. This is achieved by anchoring the dependency to onnxruntime-web@1.14.0 or @1.22.0 and explicitly importing the lower-level ort-wasm tier rather than relying on the default ort-wasm-simd or ort-wasm-simd-threaded execution paths.¹⁷

Concrete Parameterization

To tune the Silero VAD specifically for Swaram's Indian PWA context, the following parameter configurations are required:

- **positiveSpeechThreshold:** Configure between 0.7 and 0.85 to ensure robust onset detection without capturing background interference.³
- **preSpeechPadFrames:** Maintain an aggressive pre-roll buffer (equivalent to 300ms, or approximately 10 frames at standard block sizes).³ This ensures that soft, unvoiced consonants common in Indic languages (e.g., the unaspirated 't' or 'k') are captured in the audio payload sent to the STT engine.
- **redemptionFrames:** This defines the number of silence frames allowed before an utterance is considered complete. In noisy Indian environments, this must be adaptive. A baseline of 14-20 frames is necessary to prevent internal pauses (e.g., a user pausing to read an Aadhaar card) from triggering a premature cutoff.¹⁶

2.2 Transitioning to Semantic Endpointing

Even with a perfectly tuned neural acoustic VAD, relying strictly on silence timeouts creates severe user experience degradation. When a user pauses to think, recall an account number, or navigate a paper form, the acoustic VAD forcefully endpoints the interaction the moment the silence threshold is crossed. This cuts the user off mid-thought, forces an error recovery loop, and erodes trust in the system.¹⁹

The paradigm shift required for 2026 voice applications is "Semantic Endpointing".²⁰ This technique shifts the burden of turn detection from acoustic silence to linguistic comprehension.

- **The Mechanism:** Semantic endpointing utilizes a highly optimized, low-latency Large Language Model (such as Groq's Llama-3.1-8b-instant, which operates at 840 tokens per second¹⁴) to evaluate the syntactic and pragmatic completeness of an utterance.²⁰
- **Execution Flow:** While the user speaks, the interim transcripts generated by the Sarvam WebSocket are continuously forwarded to the local evaluation logic. When the acoustic VAD registers a brief silence (e.g., 200ms), the application does not immediately terminate the recording. Instead, it queries the LLM: *"The user is being asked for a 10-digit PAN. The current transcript is: 'My PAN number is A B C P...'. Is this response semantically complete?"*
- **Dynamic Adjustment:** Because the LLM understands that the sequence requires ten characters and only four have been provided, it returns a low probability of completion. The Swaram application subsequently overrides the acoustic VAD, dynamically extending the silence timeout to 3000ms.²² This grants the user the necessary time to read the document without interruption. If the LLM determines the utterance is grammatically and contextually complete, the client immediately transmits a {"flush_signal": true} to the STT engine to terminate the turn.³

3. Proper-Noun Accuracy: The State of the Art in Hypothesis Fusion

The most critical quality bottleneck identified in the Swaram dossier is the accurate transcription of Indian proper nouns (names, cities, localized entities). Single-engine architectures inherently struggle with this due to the long-tail phonetic diversity of the

subcontinent. Swaram's current approach utilizes a proprietary N-best ensemble and LLM-driven deduplication strategy. However, the current methodology relies on rigid heuristics rather than advanced neural synthesis.

3.1 Exploiting Keyterm Biasing Mechanisms

An immediate avenue for accuracy enhancement is the deployment of dynamic, request-level keyterm biasing. This technique explicitly feeds contextual vocabulary to the acoustic model prior to decoding, forcing the statistical weights toward domain-specific terminology.

Azure PhraseListGrammar Integration The Azure Speech SDK provides a highly sophisticated biasing mechanism through the PhraseListGrammar class.²³ This capability allows the application to inject arbitrary arrays of strings directly into the recognizer's memory buffer before an audio stream is processed.

- **Implementation:** Within the Next.js proxy, the server instantiates the Azure recognizer and calls `PhraseListGrammar.fromRecognizer(recognizer)`.²³ It then executes the `addPhrases()` method, passing the entire user-specific `knownNames` array extracted from the local profile, along with the specific form field label.²³
- **Weight Modification:** Crucially, the Azure SDK permits manipulation of the bias weight via the `setWeight(number)` method, which accepts values between 0.0 and 2.0 (defaulting to 1.0).²³ By elevating this weight to 1.5 during critical proper-noun fields, the system heavily penalizes generic English homophones in favor of the stored Indian nomenclature, effectively forcing the engine to select the localized entity. Note that this feature is strictly supported in the real-time online SDK, not the batch transcription APIs.²⁵

Vendor Disqualifications for Biasing While other vendors market biasing capabilities, they are largely unsuitable for Swaram's specific constraints. AssemblyAI offers a `keytermsPrompt` feature allowing up to 100 domain-specific words.²⁷ Deepgram similarly provides a `keywords` parameter.²⁹ However, as demonstrated in Section 4 of this report, both AssemblyAI and Deepgram exhibit catastrophic baseline error rates on Dravidian languages, rendering their advanced biasing features moot for the Swaram ecosystem.³⁰ Sarvam AI, despite its regional excellence, does not currently support hotword or phrase-list biasing on its Saaras v3 STT endpoints.³¹

3.2 Transitioning to Generative Error Correction (GER)

Swaram's current fusion LLM operates on a strict paradigm: it dedupes hypotheses, rejects rambling, and refuses to synthesize characters absent from the raw STT inputs. The academic frontier, represented by frameworks such as "RobustGER" and "Whispering-LLaMA," proves that this rigid approach is suboptimal. LLMs achieve drastically higher accuracy when performing *Generative Error Correction* (GER), where they are permitted to infer and reconstruct intent from noisy acoustic approximations.³²

- **The Whispering-LLaMA Paradigm:** Research indicates that passing an N-best hypothesis list into a sufficiently large language model, guided by carefully constructed context prompts, yields a Word Error Rate Relative Reduction (WERR) of up to 37.66%

compared to simple reranking methods.³⁴

- **Application to the Swaram Ensemble:** The Groq Llama-3.3-70b fusion model must be repurposed to execute a cross-modal phonetic analysis. When presented with divergent hypotheses—for example, Sarvam transcribing "Twinsha Tilkan" and Azure proposing "Tvinsha T Thilakan"—the LLM must be instructed to map these variations phonologically rather than orthographically. The prompt must inject the knownNames dictionary and instruct the LLM: *"The provided hypotheses represent noisy acoustic interpretations of an Indian name. Identify the underlying phonetic intent and map it to the closest valid entity within the known names list."*
- **Phonetic Transliteration Injection:** To further aid the LLM, the Next.js server should utilize the existing Sarvam transliteration API proxy to convert the Latin knownNames into their native Indic script (e.g., Hindi or Malayalam) and inject both the Latin and native scripts into the prompt. LLMs possess deep multilingual latent spaces; providing the native script helps the model disambiguate phonemes that are lost when forced into the English alphabet.

3.3 Activating the Gemini 3.1 Flash-Lite Audio Pass

The dossier notes a dormant code path designed for a Gemini audio-native pass. With the recent release of Google's Gemini 3.1 Flash-Lite model, activating this fourth ensemble arm is highly recommended.

- **Capabilities:** Gemini 3.1 Flash-Lite is specifically optimized for automated speech recognition tasks and low-latency, high-volume workloads.³⁷ It natively processes multimodal input, allowing it to directly ingest raw audio without relying on an intermediate text transcript.³⁷
- **Cost Efficiency:** The model is priced aggressively at \$0.50 per 1 million input tokens for audio (equivalent to approximately \$0.0053 per minute of audio), making it highly viable for the targeted 2-10 second clips utilized in the accuracy ensemble.³⁹
- **Contextual Multimodal Prompting:** Traditional STT engines are deterministic and unaware of conversational state. Gemini 3.1 Flash-Lite allows Swaram to pass the audio blob alongside a highly specific text prompt: *"Listen to this audio clip. The user is a resident of Kerala answering a form field labeled 'Full Name'. Known valid names associated with this profile include: [Array of Names]. Transcribe the exact name spoken in the audio."* This approach fuses acoustic analysis directly with semantic biasing in a single pass, often circumventing the structural failures of standard speech-to-text decoders.

4. Indian-Language and Code-Switching STT Landscape

Optimizing the everyday transcription path and composing the ideal ensemble requires a ruthless, evidence-based assessment of vendor performance across the vast linguistic diversity of the Indian subcontinent. Vendor marketing claims regarding multilingual support frequently collapse when confronted with real-world Indian accents, code-mixed Hinglish, and

regional dialects.

4.1 Insights from the "Voice of India" 2026 Benchmark

The definitive empirical baseline for evaluating STT performance in this context is the 2026 "Voice of India" benchmark (arXiv:2604.19151).³⁰ This large-scale study analyzed 536 hours of unscripted, telephonic speech across 15 major Indian languages, specifically testing robustness against spelling variations, audio degradation, and geographical dialects.³⁰ The benchmark results expose critical vulnerabilities in major global models while validating the efficacy of localized architectures.

Model / Vendor	Overall Indic Assessment	Specific Language Performance Notes	Tier Classification
Sarvam Audio	Exceptional (8.2% WER)	Achieved the lowest Word Error Rate in 13 out of 15 evaluated languages. Dominant across the Hindi belt and Bengali. ⁴¹	Tier I
Gemini 3 Pro	Exceptional (6.9% WER)	Demonstrated unparalleled robustness on highly dialectal variations such as Bhojpuri and Chhattisgarhi. ⁴¹	Tier I
Groq Whisper (GPT-4o architecture)	Highly Variable	Capable on Hindi, but exhibits catastrophic degradation on Gujarati (295.9% WER) and Malayalam (167.8% WER) due to generative hallucination loops triggered by script	Tier III (for Dravidian/Gujarati)

		diversity. ³⁰	
Deepgram Nova 3	Substandard (16.7% WER)	Moderate performance on Indo-Aryan languages, but yields severely elevated error rates on Tamil (67.8%) and Odia (89.8%). ³⁰	Tier III
AssemblyAI Universal	Complete Failure (>100% WER)	Driven by failed language detection and systemic hallucinations, error rates routinely exceed 100%, indicating complete transcription failure on Indic speech. ³⁰	Tier III
ElevenLabs Scribe v2	Fragile	Highly susceptible to audio degradation. WER spikes from 15.31% to 25.20% on lower-quality audio quartiles. ⁴¹	Tier II

4.2 Resolving Dossier Assumptions and Architecture Alignment

Critical Contradiction Flag: The Swaram dossier (§6.1, §6.2) explicitly designates Groq Whisper large-v3-turbo and on-device Whisper models as the universal fallback mechanisms for all languages. The Voice of India benchmark definitively proves that Whisper architectures are mathematically incapable of reliably transcribing unscripted Malayalam or Gujarati, entering catastrophic hallucination loops where error rates approach 300%.³⁰ **Whisper must be immediately excised from the fallback chain for any UI session operating in a Dravidian language (ml-IN) or Gujarati.**

Everyday Path Composition: Based on the empirical data, the single-engine "everyday" path is currently correctly assigned to Sarvam Saaras v3. Operating at Tier I efficiency, it provides the most consistent baseline accuracy across both the Indo-Aryan and Dravidian language families.³⁰ For Malayalam (ml-IN) users experiencing a Sarvam failure, the fallback route must be redirected to Azure Speech REST or the native Web Speech API, entirely bypassing the

Whisper infrastructure to prevent hallucination cascades.

Code-Switching Capabilities: Swaram's target demographic heavily utilizes Hinglish and code-mixed vocabulary. Standard STT engines often attempt to force English loanwords into phonetic native scripts, destroying formatting. Sarvam's saaras:v3 model features a dedicated `mode=codemix` query parameter accessible via the WebSocket connection.³ When enabled, the model intelligently outputs code-mixed text, retaining the native Indic script for regional words while preserving Latin script for English terminology (e.g., "मेरा phone number है 9840950950").³ Swaram's downstream transcriptNormalizer (§7.1) must be updated to process this interleaved, multi-script output accurately.

5. Personalization and Context Carrying

State-of-the-art voice agents derive significant accuracy improvements by maintaining deep conversational context across turns, ensuring the acoustic model is heavily biased toward the immediate topic of conversation. Personalization in Swaram must extend beyond the static nameDictionary (§7.3) to embrace dynamic context injection.

Architectural Approaches for Context Carrying:

1. **Cascading Geographical Priors:** As a user traverses a form, previously confirmed values must act as probabilistic priors for subsequent fields. For example, if a user confirms their "State" as "Kerala" and their "District" as "Ernakulam," these entities must be temporarily cached in the session state. When the system subsequently asks for a "City" or "Tehsil," the Next.js server must inject all known municipalities within Ernakulam directly into the Azure PhraseListGrammar and the Groq LLM fusion prompt. This drastically narrows the acoustic search space for the recognizer.
2. **Cross-Engine Context Injection:** The entire serialized array of confirmed form fields must flow seamlessly into every analytical node. For the Gemini 3.1 Flash-Lite audio pass, the session context (formatted as a concise JSON object) must be appended to the prompt instruction block. This mirrors the mechanics utilized by ChatGPT Voice and Siri, where the LLM utilizes the historical dialogue state to disambiguate identical phonemes based on semantic probability.

6. Hallucination Mitigation and Robustness Frameworks

The vulnerability to Whisper hallucinations on short, clipped audio, combined with the lack of graceful degradation during vendor outages, represents a significant risk to the Swaram user experience (§11 - W10, W11).

6.1 Suppressing Hallucinations Without Logprobs

Because the Groq API does not expose the granular token-level `no_speech_prob` or text compression ratio metrics native to the Whisper architecture, the engineering team must implement alternative, upstream heuristics to detect and suppress generative hallucinations.

- **VAD-Driven Audio Quality Gating:** The most effective method to prevent Whisper from hallucinating over static is to prevent the static from reaching the model. The client-side

Silero VAD must execute a pre-flight check before transmitting the audio blob. By calculating the ratio of positive speech frames to negative silence frames within a recorded segment, the client can determine if the utterance was a genuine vocalization or an accidental noise transient that breached the threshold. If an utterance is comprised of >85% silence frames, it must be discarded locally, returning an empty string without burning a network request.

- **LLM Entropy and Repetition Detection:** When Whisper hallucinates on noisy inputs, it frequently emits repetitive, low-entropy sequences (e.g., "Thank you. Thank you. Thank you." or endlessly repeating a single phoneme). The correctTranscript fast lane (powered by Groq Llama-3.1-8b-instant) should be augmented with a specific system directive to calculate the entropy of the transcript. If the string exhibits cyclical hallucination patterns that lack semantic coherence to the requested form field, the LLM must be instructed to scrub the transcript and return an empty string "", forcing the state machine to politely ask the user to repeat the input.

6.2 Graceful Vendor-Outage Degradation

Swaram currently relies on a reactive, per-clip 3-retry fallback mechanism. During a widespread Sarvam or Azure outage, every single user interaction suffers a massive latency penalty as the client repeatedly waits for timeouts before falling back.

- **The Global Circuit Breaker Pattern:** The Next.js /api/transcribe route must implement a stateful, global circuit breaker using an in-memory store (or Redis, if scaled). If the primary vendor (Sarvam) returns HTTP 500 errors or exceeds a 3000ms timeout threshold across 5 consecutive global requests, the server must "trip" the circuit breaker.
- **Stateful Routing:** Once tripped, the server immediately and automatically shifts all incoming transcription requests to the secondary cloud provider (Azure) for a predefined cooldown period (e.g., 5 minutes). This ensures that a localized vendor outage causes, at most, a few seconds of disruption to a handful of users, rather than degrading the entire application's responsiveness. The circuit periodically allows a "half-open" test request to check if the primary vendor has recovered before normalizing routing.

7. Evaluation Harness: Establishing an Empirical Baseline

The Swaram engineering team currently operates without a systematic measurement harness, meaning architectural changes to the ensemble or prompt logic cannot be quantitatively evaluated for regressions (§11 - W13). Building a localized benchmarking suite is mandatory.

Designing the Swaram Benchmark Test Set:

1. **Dataset Construction:** The team must curate a proprietary golden dataset modeled directly on the methodologies of the Voice of India paper. This involves recording 500 distinct audio clips (ranging from 2 to 10 seconds) of Indian names, localized addresses, and alphanumeric identifiers (PAN, Aadhaar). The dataset must encompass diverse speakers, explicitly balancing geographic dialects and gender, as models historically penalize male speakers and out-of-region migrants.³⁰

2. **Synthetic Augmentation:** To stress-test the acoustic boundaries of the STT engines, utilize Sarvam Bulbul v3 and Azure Neural TTS to synthetically generate an additional 500 clips of edge-case, highly complex nomenclature.
3. **Orthographically-Informed Evaluation Metrics:** Standard Word Error Rate (WER) is fundamentally broken for Indian languages because it harshly penalizes valid orthographic variations. For instance, if an STT engine outputs "Tvinsha" instead of "Twinsha", strict WER registers a complete failure, despite both being phonetically accurate transliterations of the same Indic name. The harness must implement the **Orthographically-Informed WER** metric utilized by the Voice of India researchers, which maps outputs against an array of valid, acceptable spellings before calculating the error distance.³⁰
4. **Entity-Accuracy Scoring:** Ultimately, form-filling relies on absolute string matches. The harness must include an Entity-Accuracy metric—a Boolean Pass/Fail evaluating whether the pipeline successfully extracted the exact PAN sequence or the normalized name, regardless of filler words.
5. **CI/CD Integration:** This evaluation suite must be bundled as a Node.js script (scripts/eval-harness.mjs) that pushes the golden audio directory through the local Next.js /api/transcribe route, generating a diff report against the verified JSON manifest to gate pull requests.

8. Text-to-Speech (TTS): Enhancing Prosody and Latency

The secondary objective of this research is optimizing the TTS pipeline, which currently suffers from high Time-To-First-Byte (TTFB) on uncached dynamic strings and exhibits poor pronunciation of Indian proper nouns during English readback sequences (§11 - W14, W15, W16).

8.1 Leveraging Sarvam Bulbul v3 Pronunciation Dictionaries (dict_id)

When Swaram attempts to read back a confirmed form field (e.g., the name "Thilakan") in English (en-IN), the Latin-to-sound rules of TTS engines routinely butcher the pronunciation. The application currently utilizes a cumbersome transliteration workaround to force Hindi/Malayalam output. Sarvam's Bulbul v3 model elegantly solves this via explicit pronunciation dictionaries requested through the dict_id parameter.⁴⁴

- **Dictionary Compilation and Registration:** The engineering team must compile a static JSON file mapping complex, frequently encountered Indian names to their desired phonetic equivalents. The required schema utilizes a language-keyed map: {"pronunciations": {"en-IN": {"Sarvam": "Saar-vum", "Thilakan": "थिलकन"}}}.⁴⁴
- **API Workflow:** This JSON payload is uploaded via a multipart/form-data POST request to the Sarvam API (<https://api.sarvam.ai/text-to-speech/pronunciation-dictionary>).⁴⁶ The API returns a unique string identifier, the dictionary_id (e.g., p_5cb7faa6).⁴⁶
- **Execution:** Within the Next.js /api/tts route, when invoking the bulbul:v3 model, this dict_id is appended to the request payload.⁴⁵ The TTS engine intercepts the text stream,

autonomously identifies words matching the dictionary keys, and swaps them with the custom phonetics prior to waveform synthesis. This guarantees flawless Indic pronunciation while maintaining the English linguistic framework, rendering the legacy transliteration hack obsolete. (Note: Sarvam enforces a strict limit of 10 dictionaries per user and 100 words per dictionary; the payload must be carefully curated to target the highest-impact failures).⁴⁷

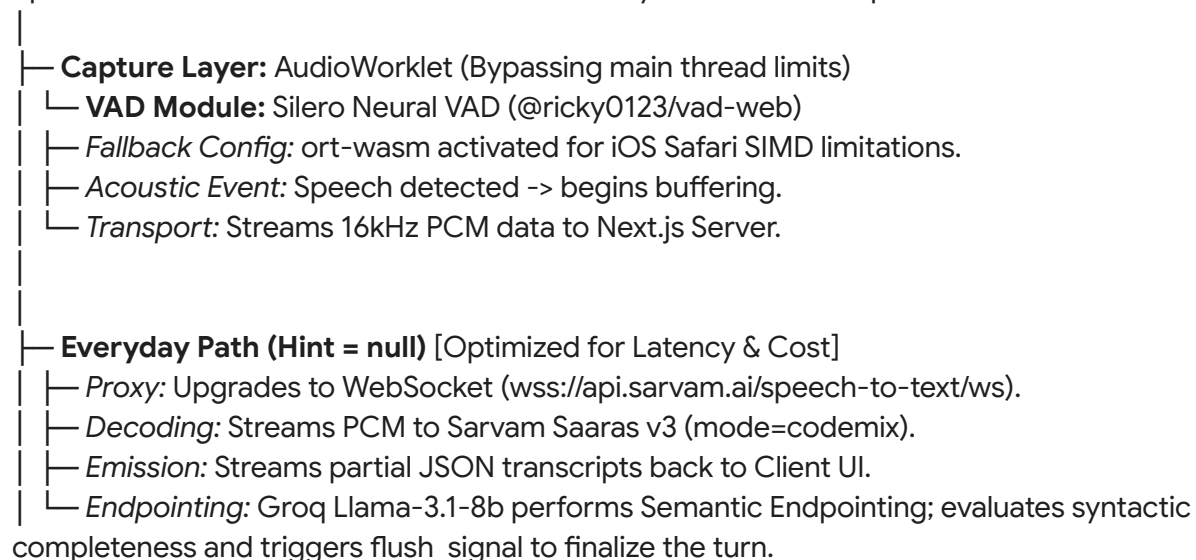
8.2 Streaming TTS for Sub-Second First-Audio Latency

To eradicate the high latency associated with downloading fully finalized MP3 blobs for uncached conversational turns, the `/api/tts` route must transition to a streaming transport.

- **Implementation:** Sarvam provides a dedicated WebSocket TTS streaming endpoint (`/text-to-speech/ws`).⁴⁹ By establishing a persistent connection, the time-to-first-byte is reduced to milliseconds, as the server begins transmitting base64-encoded audio chunks the moment the first tokens are processed.⁴⁹
- **Client Playback:** The Next.js client can intercept these chunks and feed them directly into the Web Audio API or a Media Source Extensions (MSE) buffer, initiating audio playback while the tail end of the sentence is still being synthesized by the cloud server.
- **Prosody Enhancements:** For the specific use case of spelled-letter readback (e.g., dictating an email address "T, W, I, N..."), Bulbul v3 lacks support for precise SSML pitch or loudness tags.⁴⁵ To simulate deliberate, rhythmic dictation spacing, the application must utilize the pace parameter. Scaling the pace down to 0.6 (within the allowed 0.5 to 2.0 range) specifically for spell-mode strings effectively lengthens the phonetic pauses, naturally emulating human dictation without requiring complex audio manipulation.⁴⁴

9. Proposed Target Architecture Diagram

The upgraded STT pipeline must cleanly separate the high-speed, cost-efficient everyday operations from the resource-intensive accuracy mechanisms required for critical form fields.



- └─ **Accuracy Path (Hint = "name" / "spell")** [Optimized for Precision]
 - | *Triggered upon finalizing a critical audio blob via REST.*
 - └─ **Parallel Engine Fan-Out:**
 - | └─ 1. Sarvam Saaras v3 (Baseline Indian English/Indic competence).
 - | └─ 2. Azure Speech REST (Generates Detailed N-best lists + applies PhraseListGrammar Biasing mapped to local knownNames).
 - | └─ 3. Gemini 3.1 Flash-Lite (Multimodal Audio Pass + Text Prompt injecting Contextual Priors).
 - | └─ 4. Groq Whisper Turbo (Skipped explicitly if UI lang == ml-IN or gu-IN to prevent hallucination cascades).
- └─ **Generative Error Correction (GER) Module:**
 - └─ *Model:* Groq LLaMA-3.3-70b.
 - └─ *Prompting:* Instructed to perform phonetic alignment across the varied N-best hypotheses and map intents to the knownNames array.
 - └─ *Output:* Emits the unified, phonetically corrected string to the dialogue state machine.

10. Migration Plan

Deploying these architectural shifts requires a phased, isolated rollout strategy to ensure application stability and allow for continuous regression testing.

Phase 1: Quick Wins and Threat Mitigation (≤ 1 Week)

- **Action 1:** Immediately remove Groq Whisper large-v3-turbo and on-device Whisper from the fallback chain for Malayalam (ml-IN) and Gujarati sessions, redirecting these pathways to Azure or the native Web Speech API to eliminate catastrophic hallucination loops identified in the Voice of India benchmark.
- **Action 2:** Activate the dormant Gemini multimodal code pass in `/api/transcribe`. Update the integration to utilize the cost-effective Gemini 3.1 Flash-Lite model, ensuring the `x-known-names` variable is injected into the accompanying text prompt for context.
- **Action 3:** Compile a JSON dictionary of the top 50 historically mispronounced Indian names. Register this via the Sarvam API to generate a `dictionary_id`, and append it to the `/api/tts` calls executing the `bulbul:v3` model to immediately resolve English readback prosody issues.

Phase 2: Core Architecture Refactoring (≤ 1 Month)

- **Action 1:** Migrate the VAD architecture from the deprecated `ScriptProcessorNode` to the `@ricky0123/vad-web` library (Silero). Implement the explicit environment check to load the `ort-wasm` fallback binary for older iOS Safari clients.
- **Action 2:** Refactor the `llm.ts` fusion module to implement Generative Error Correction (GER). Rewrite the system prompt to encourage phonetic mapping across the N-best hypotheses rather than applying strict, programmatic deduplication.
- **Action 3:** Integrate the Azure SDK's `PhraseListGrammar` within the Accuracy Path, dynamically mapping the user's localized `knownNames` array and form field context to the recognizer prior to audio ingestion.

Phase 3: Ambitious Streaming Overhaul (1–3 Months)

- **Action 1:** Replace the existing clip-based, REST-driven everyday transcription mechanism with the Half-Cascade Streaming Architecture. Build the Next.js WebSocket proxy to manage persistent connections with the Sarvam Saaras v3 endpoint.
- **Action 2:** Implement Semantic Endpointing. Introduce the fast Groq Llama-3.1-8b-instant model into the loop to evaluate the syntactic completeness of interim transcripts, allowing the application to dynamically override acoustic silence timeouts and prevent premature utterance cutoffs.
- **Action 3:** Construct and deploy the comprehensive Evaluation Harness. Curate the localized audio test set and implement the Orthographically-Informed WER metric to establish an automated CI/CD gating mechanism, ensuring that future prompt modifications or model upgrades are mathematically validated against true Indian speech diversity before reaching production.

Works cited

1. Sequential Pipeline Architecture for Voice Agents - LiveKit, accessed on July 19, 2026, <https://livekit.com/blog/sequential-pipeline-architecture-voice-agents>
2. Voice Agent Architecture: STT, LLM, and TTS Pipelines Explained | LiveKit, accessed on July 19, 2026, <https://livekit.com/blog/voice-agent-architecture-stt-llm-tts-pipelines-explained>
3. WebSocket | Sarvam API Docs, accessed on July 19, 2026, <https://docs.sarvam.ai/api-reference/speech-to-text/transcribe/ws>
4. API Pricing - Sarvam AI, accessed on July 19, 2026, <https://www.sarvam.ai/api-pricing>
5. Pricing | Sarvam API Docs, accessed on July 19, 2026, <https://docs.sarvam.ai/api/pricing>
6. How to use compressed input audio - Speech service, accessed on July 19, 2026, <https://docs.azure.cn/en-us/ai-services/speech-service/how-to-use-codec-compressed-audio-input-streams>
7. Streaming Audio from Azure Communication Service Outbound Call to Cognitive Service, accessed on July 19, 2026, <https://learn.microsoft.com/en-my/answers/questions/2283300/streaming-audio-from-azure-communication-service-o>
8. Azure Speech in Foundry Tools pricing, accessed on July 19, 2026, <https://azure.microsoft.com/en-us/pricing/details/speech/>
9. Enterprise Speech-to-Text: Cost vs Benefits, accessed on July 19, 2026, <https://blog.naive.cloud/enterprise-speech-to-text-cost-vs-benefits/>
10. Pricing and usage of Pronunciation Assessment feature from Azure Speech-to-Text, accessed on July 19, 2026, <https://learn.microsoft.com/en-us/answers/questions/5608069/pricing-and-usage-of-pronunciation-assessment-feat>
11. Speech to Text Overview - Speech Service - Foundry Tools - Microsoft Learn, accessed on July 19, 2026,

- <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/speech-to-text>
12. Groq Pricing In 2026: Every Model, Tier, And Cost Compared - CloudZero, accessed on July 19, 2026, <https://www.cloudzero.com/blog/groq-pricing/>
 13. Whisper Large v3 Turbo - GroqDocs - Groq Console, accessed on July 19, 2026, <https://console.groq.com/docs/model/whisper-large-v3-turbo>
 14. Groq On-Demand Pricing for Tokens-as-a-Service, accessed on July 19, 2026, <https://groq.com/pricing>
 15. ricky0123/vad: Voice activity detector (VAD) for the browser with a simple API - GitHub, accessed on July 19, 2026, <https://github.com/ricky0123/vad>
 16. VAD-WEB error on safari : SIMD not supported in the current environment #157 - GitHub, accessed on July 19, 2026, <https://github.com/ricky0123/vad/issues/157>
 17. WASM Memory Out of Range Error on iPhone Devices · Issue #134 · ricky0123/vad - GitHub, accessed on July 19, 2026, <https://github.com/ricky0123/vad/issues/134>
 18. vad | Flutter package - pub.dev, accessed on July 19, 2026, <https://pub.dev/packages/vad>
 19. Semantic VAD for Voice Agents: Turn Detection 2026 - Gradium, accessed on July 19, 2026, <https://gradium.ai/content/semantic-vad-voice-agents-turn-detection-2026>
 20. Semantic End-of-Turn Detector - Emergent Mind, accessed on July 19, 2026, <https://www.emergentmind.com/topics/semantic-end-of-turn-detector>
 21. Semantic VAD: turn detection that uses meaning, not silence - Gradium, accessed on July 19, 2026, <https://gradium.ai/blog/semantic-vad>
 22. anyone have experience with turn detection for communication between humans and AI agents? : r/LocalLLaMA - Reddit, accessed on July 19, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1py94k8/anyone_have_experience_with_turn_detection_for/
 23. PhraseListGrammar class - Microsoft Learn, accessed on July 19, 2026, <https://learn.microsoft.com/en-us/javascript/api/microsoft-cognitiveservices-speech-sdk/phaselistgrammar?view=azure-node-latest>
 24. microsoft-cognitiveservices-speech-sdk package, accessed on July 19, 2026, <https://learn.microsoft.com/en-us/javascript/api/microsoft-cognitiveservices-speech-sdk/?view=azure-node-latest>
 25. Custom list phrase / vocabulary on batch transcriptions? - Microsoft Learn, accessed on July 19, 2026, <https://learn.microsoft.com/en-us/answers/questions/360649/custom-list-phrase-vocabulary-on-batch-transcripti>
 26. Custom list phrase / vocabulary on batch transcriptions? - Microsoft Q&A, accessed on July 19, 2026, <https://learn.microsoft.com/en-my/answers/questions/360649/custom-list-phrase-vocabulary-on-batch-transcripti>
 27. AssemblyAI Provider - AI SDK, accessed on July 19, 2026, <https://ai-sdk.dev/providers/ai-sdk-providers/assemblyai>
 28. Vapi voice agent with AssemblyAI Universal-3 Pro Streaming, accessed on July 19, 2026,

<https://www.assemblyai.com/blog/vapi-voice-agent-with-assemblyai-universal-3-pro-streaming>

29. Word Boosting in AI Transcription: How to Fix Product Names and Rare Vocabulary, accessed on July 19, 2026, <https://www.mindstudio.ai/blog/word-boosting-ai-transcription-custom-vocabulary>
30. Voice of India: A Large-Scale Benchmark for Real-World Speech Recognition in India - arXiv, accessed on July 19, 2026, <https://arxiv.org/html/2604.19151v2>
31. ThinnestAI vs Sarvam AI (2026): Use Both — Here's When, accessed on July 19, 2026, <https://thinnest.ai/compare/thinnestai-vs-sarvam-ai>
32. Generative Error Correction for Emotion-aware Speech-to-text Translation - ACL Anthology, accessed on July 19, 2026, <https://aclanthology.org/2025.findings-acl.1047/>
33. YUCHEN005/RobustGER: Code for paper "Large Language Models are Efficient Learners of Noise-Robust Speech Recognition" - GitHub, accessed on July 19, 2026, <https://github.com/yuchen005/robustger>
34. Whispering LLaMA: A Cross-Modal Generative Error Correction Framework for Speech Recognition | OpenReview, accessed on July 19, 2026, <https://openreview.net/forum?id=OETPPc15XG-eld=vvMikg1hl2>
35. Listening and Seeing Again: Generative Error Correction for Audio-Visual Speech Recognition - arXiv, accessed on July 19, 2026, <https://arxiv.org/html/2501.04038v1>
36. Whispering LLaMA: A Cross-Modal Generative Error Correction Framework for Speech Recognition - ACL Anthology, accessed on July 19, 2026, <https://aclanthology.org/2023.emnlp-main.618.pdf>
37. Gemini 3.1 Flash-Lite | Gemini Enterprise Agent Platform - Google Cloud Documentation, accessed on July 19, 2026, <https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/gemini/3-1-flash-lite>
38. Gemini 3.1 Flash-Lite - Google DeepMind, accessed on July 19, 2026, <https://deepmind.google/models/gemini/flash-lite/>
39. Gemini Developer API pricing, accessed on July 19, 2026, <https://ai.google.dev/gemini-api/docs/pricing>
40. Voice of India: A Large-Scale Benchmark for Real-World Speech Recognition in India - arXiv, accessed on July 19, 2026, <https://arxiv.org/abs/2604.19151>
41. Voice of India: A Large-Scale Benchmark for Real-World Speech Recognition in India - arXiv, accessed on July 19, 2026, <https://arxiv.org/html/2604.19151v1>
42. A Large-Scale Benchmark for Real-World Speech Recognition in India - arXiv, accessed on July 19, 2026, <https://arxiv.org/pdf/2604.19151>
43. Paper Digest: Recent Papers on Speech Recognition, accessed on July 19, 2026, <https://www.paperdigest.org/2020/06/recent-papers-on-speech-recognition/>
44. Bulbul | Sarvam API Docs, accessed on July 19, 2026, <https://docs.sarvam.ai/api/getting-started/models/bulbul>
45. REST | Sarvam API Docs, accessed on July 19, 2026, <https://docs.sarvam.ai/api-reference/text-to-speech/convert>

46. Create | Sarvam API Docs, accessed on July 19, 2026,
<https://docs.sarvam.ai/api-reference/pronunciation-dictionary/create>
47. Create | Sarvam API Docs, accessed on July 19, 2026,
<https://docs.sarvam.ai/api/pronunciation-dictionary/create>
48. Pronunciation Dictionary | Sarvam API Docs, accessed on July 19, 2026,
<https://docs.sarvam.ai/api-reference-docs/api-guides-tutorials/text-to-speech/pronunciation-dictionary>
49. HTTP Streaming API - Sarvam AI Docs, accessed on July 19, 2026,
<https://docs.sarvam.ai/api/api-guides-tutorials/text-to-speech/streaming-api/http-stream>
50. Streaming Text-to-Speech API - Sarvam AI Docs, accessed on July 19, 2026,
<https://docs.sarvam.ai/api/api-guides-tutorials/text-to-speech/streaming-api/web-socket>